

LAB REPORT 03

RELATIONAL OPERATORS, LOGICAL OPERATORS, AND CONDITIONALS

1. OBJECTIVE

- To understand and apply comparison/relational operators.
- To utilize logical operators for complex condition checking.
- To implement decision-making using if, if-else, and else-if statements.

2. THEORETICAL BACKGROUND

Relational Operators. Relational operators are used to compare two values. They return a boolean result (true or false). These include less than (<), greater than (>), less than or equal to (<=), greater than or equal to (>=), equal to (==), and not equal to (!=).

Logical Operators. Logical operators allow for the combination of multiple conditions. The NOT operator (!) inverts a boolean value, the AND operator (&&) returns true only if both conditions are true, and the OR operator (||) returns true if at least one condition is true.

Decision Control Structures. By default, programs execute sequentially. Decision control structures allow the program to branch based on conditions. The 'if' statement executes a block if a condition is true, 'if-else' provides an alternative block if false, and 'else-if' allows for cascading checks of multiple conditions.

3. SOFTWARE USED

- IDE: DEV-C++ IDE / VS Code
- Compiler: C++ Compiler

4. EXPERIMENTAL TASKS

Task 1: Relational Operators

Write a C++ program that prompts the user to enter two numbers. Use comparison operators to check if the first number is equal to, greater than, or less than the second number.

```
#include <iostream>
using namespace std;

int main() {
    int num1, num2;
    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    if (num1 == num2) cout << num1 << " is equal to " << num2 << endl;
    else if (num1 > num2) cout << num1 << " is greater than " << num2 <<
endl;
    else cout << num1 << " is less than " << num2 << endl;

    return 0;
}
```

Task 2: Logical Operators

Write a C++ program that prompts the user to enter an integer. Use logical operators to check if the entered number is greater than 0 and less than 100.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin >> num;

    if (num > 0 && num < 100) {
        cout << "The number is between 0 and 100." << endl;
    } else {
        cout << "The number does not meet the criteria." << endl;
    }
    return 0;
}
```

Task 3: If Statement

Write a C++ program that prompts the user to enter an integer and uses an if statement to check if it is positive.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin >> num;

    if (num > 0) {
        cout << "The number is positive." << endl;
    }
    return 0;
}
```

Task 4: If-Else Statement

Write a C++ program that prompts the user for their age. Use an if-else statement to check if the age is greater than or equal to 18.

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;

    if (age >= 18) cout << "You are eligible to vote." << endl;
    else cout << "You are not eligible to vote." << endl;

    return 0;
}
```

Task 5: Else-If Statement

Write a C++ program that prompts the user to enter an integer and uses an else-if structure to check if the number is positive, negative, or zero.

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin >> num;

    if (num > 0) cout << "Positive." << endl;
    else if (num < 0) cout << "Negative." << endl;
    else cout << "Zero." << endl;

    return 0;
}
```

5. CONCLUSION

This lab focused on the decision-making capabilities of C++. By utilizing relational and logical operators, we successfully created programs that can evaluate complex conditions. The implementation of if, if-else, and else-if structures allowed for controlled branching in program flow, which is a fundamental aspect of creating responsive and logic-based software applications.